2025 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W) | 979-8-3315-1205-7/25/$31.00 ©2025 IEEE | DOI: 10.1109/DSN-W65791.2025.00050

A Margin Based Early-Stopping Approach for Random Forest classifiers

Mario Barbareschi*, Antonio Emmanuele*
*Department of Electrical Engineering and Information Technologies, University of Naples Federico II, 80125, Naples, Italy
Email: {name.surname}@unina.it

***Abstract*—In recent years, the continuous expansion of the Internet of Things (IoT) has driven a shift toward decentralized computation to improve privacy and reduce latency. As a result, computational tasks are increasingly executed directly on edge nodes within the IoT infrastructure. This includes the deployment of various Machine Learning models, such as Random Forest (RF) classifiers, which are widely used for their efficiency and interpretability. However, edge nodes are typically constrained in terms of computational power and memory, making it difficult to achieve high throughput when running complex RF models. Additionally, the large size of these models leads to higher energy consumption during inference, which can significantly impact the battery life of edge devices. To address these challenges, we propose an early-stopping mechanism for RF classifiers. The method consists of an offline phase, where statistical measures are precomputed to estimate the confidence level of predictions, and an online phase, where inference can be stopped early based on the computed confidence.**

***Index Terms*—approximate computing, ensemble learning, Random Forests, early stopping**

I. Introduction

In recent years, the rapid development of the Internet of Things (IoT) paradigm has led to an increased use of Machine Learning (ML) models to solve various tasks, ranging from healthcare [1] to on-board passenger flow prediction [2]. However, the necessity of fast and flexible services is leading to the increased deployment of various computational tasks, including ML, directly on nodes at the edge of the IoT infrastructure. While ensuring reduced latency and better privacy, the adoption of ML models at the edge brings several challenges [3]. In particular, edge-nodes are usually low-power microcontrollers, facing limitations in computing and memory capabilities, as well as strict energy constraints when battery-powered [4].

A possible solution to these challenges consists of using models with a computationally lighter inference, which well fits the nature of edge-nodes. For these reasons, recently Decision Trees (DTs)-based models have received much attention due to their capabilities of being accurate classifiers while enabling a computationally light inference phase [5]. Indeed, differently from Deep Learning models that require complex mathematical operations, performing an inference for a DT consists in a tree-visiting procedure that involves simple comparisons of input-feature values against fixed thresholds learned during training [6].

Moreover, since DTs are prone to overfitting, they are often combined in ensembles, such as RFs. The inference of such models consists of visiting all the DTs in the ensemble and then combining their inferences to obtain the final result. Unfortunately, with the increasing complexity of datasets for IoT, the number of trees in an ensemble grows accordingly. Moreover, the elevated number of different patterns in real-world IoT training datasets leads to deeper trees. Ultimately, this results in larger models that require more time and energy to perform an inference, jeopardizing the adoption of Decision Tree ensembles in the IoT.

However, despite the increasing complexity of classifiers, not all of their inference power is required to perform accurate classifications [7]. Indeed, it has been proven that several components of such models are often redundant and can therefore be safely removed without compromising accuracy. For this reason, RF models benefits from the application of Approximate Computing (AxC) techniques, which aim to trade the accuracy of the classifier off for its computational requirements [8].

In light of the above, in this work, we propose an early-stopping criterion for RF classifiers to minimize the inference time. In particular, given a trained classifier, our approach follows two different phases: an offline phase and an after-deployment phase. During the offline phase: i) DTs are ranked in order to identify the trees most contributing to the inference; ii) statistical parameters required to evaluate the minimum number of trees for an inference are computed. Once deployed, trees are visited based on their ranking order and the confidence level computed; when the minimum number of trees for an inference is visited, then the inference is stopped. The experimental result proves the effectiveness of technique on publicly available datasets, leading to evaluate about a half of DTs of the forest paying-off 2% accuracy loss.

II. Technical Background & Related Work

A. Decision Tree Based Models

DTs are ML models represented as full binary trees, where each node is either an internal or a leaf node [9]. The internal nodes implement a specific decision rule, which consists in a comparison of a fixed input feature against a threshold value, both learned during training. Conversely, leaf nodes represent

Authors contributed equally and are listed in alphabetical order.

2325-6664/25/$31.00 ©2025 IEEE
DOI 10.1109/DSN-W65791.2025.00050

Authorized licensed use limited to: POLITECHNIKI WARSZAWSKIEJ. Downloaded on August 21,2026 at 17:14:48 UTC from IEEE Xplore. Restrictions apply.

prediction outcomes, which are continuous values in the case of regression problems, and categorical for classification problems. As mentioned above, inference for a decision tree consists of a tree traversal. Starting from the root node, an input feature is compared against a threshold. In the case of a positive comparison, the feature moves to the right child of the current node, while in the case of a negative outcome, the feature moves to the left child of the current node. This process is repeated until a leaf is reached; at this point, the inference ends with the leaf value as the prediction result.

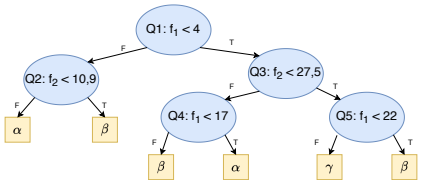


Fig. 1: Example of a Decision Tree Model

In order to reduce overfitting, RF classifier, train multiple DTs by sub-sampling for each of them both the training-samples and their features [9]. During inferences, each tree is visited iteratively, and then the results are combined. For instance, for classification problems, once each tree is visited, a vector of votes is constructed $v = [v_{c_1}, v_{c_2}, ..v_{c_M}]$. This vector contains, for each of the M classes c, the number of trees that labeled the input sample as c. The vector v is then submitted to a majority voter module, which outputs the class having the largest number of votes. In other words, the predicted class is $c_j, j = argmax(v)$.

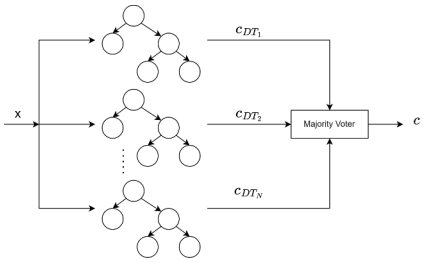


Fig. 2: Representation of the RF inference. During classification, multiple Decision Trees outputs their class and a majority voter establishes the most-popular one.

B. Margin of Random Forest

Due to the nature of majority voting, most trees may not be taken into account during the classification without altering the outcome. This assumption is valid as long as the majority voting condition holds true [3]. Indeed, the number of redundant trees during the classification of a sample x can be estimated by leveraging the margin metric [10]: $M_x = \frac{v_c - v_{c^*}}{N}$. In particular, v_c refers to the number of votes received from the correct class, v_{c^*} to the number of votes of the most-voted class different from c, and N is the number of DTs. For this reason, the absolute value of the margin indicates the strength of a classification and, thus indirectly, the number of redundant trees. A magnitude closer to 1 reflects greater confidence in the classification outcome, implying the presence of many redundant DTs. Moreover, this metric ranges from $[-1, 1]$, with negative values indicating misclassifications and positive values indicating correct classifications.

Due to its effectiveness in capturing redundancy, this value has been widely employed for ensemble pruning techniques, which aim to optimize a tree ensemble by reducing the number of decision trees. For instance, the authors of [10] propose the MeanD-M algorithm, which, for each tree, evaluates the average reduction in terms of margin. After ranking the trees, some of ones having the lowest rank — i.e., the ones that impact the least on classifications — are then pruned from the ensemble.

C. DTs at the edge

Implementations of RFs onto both Field Programmable Gate Arrays (FPGAs) and Microcontroller Unit (MCU) offer the best performance and energy trade-off when compared to General Purpose - Graphic Processing Units (GP-GPUs) due to the *all to all* communication cost. With regard to FPGAs implementations, the authors of [11] propose a speculative model-specific accellerator for Decision Tree based models. Conversely, the approaches proposed in [12] and [13], entail a custom processor implemented in FPGA that performs the tree visiting.

As for MCU, the authors of [5] propose three optimized kernels for tree-visiting on RISC-V MCUs. Differently, the authors of [14] introduce a tree-visiting procedure based on SIMD instructions.

Despite the lightness of their inference phase, the deployment of DTs based ensembles at the edge is still a challenging task due to limited computational and memory capabilites of edge-nodes. In order to deploy DTs on memory-constrained microcontrollers, the authors of [15] propose to store the entire model in the FLASH memory instead of SRAM. Differently, the authors of [3] propose to directly reduce model size by leveraging functional-approximation. The latter involves the direct removal of computational logic used to generate outputs.

III. Early Stopping for RF classifiers

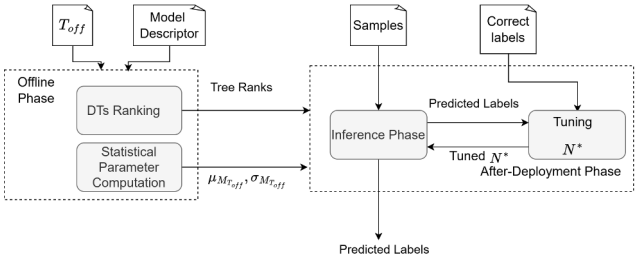


Fig. 3: Workflow of the proposed methodology.

In order to minimize the latency of inference, in this section we propose an early stopping criterion for RF classifiers. In

Authorized licensed use limited to: POLITECHNIKI WARSZAWSKIEJ. Downloaded on August 21,2026 at 17:14:48 UTC from IEEE Xplore. Restrictions apply.

particular, our solution leverages the margin metric to visit only the most significant trees during inference, avoiding unnecessary computations. To this end, our proposal, depicted in Figure 3, consists of two distinct phases: an offline phase and an online (after-deployment) phase.

A. Offline Phase

To correctly apply an early-stopping criterion at runtime, two preliminary tasks must be performed before deploying the model. We refer to the combination of these subsequent tasks as the offline phase of our proposal. Conceptually, this phase takes as input a trained RF model and produces all the information required to enable efficient early stopping after deployment.

The first task consists of ranking all the DTs based on their impact on predictions. This step is fundamental, as early stopping aims to visit the most significant trees first to ensure accurate classification. The ranking of DTs can be performed using several approaches. One option is a similarity-based ranking, which prioritizes visiting the most dissimilar trees first. Alternatively, an accuracy-based approach ranks DTs according to their performance on a portion of the test set, ensuring that the most accurate trees are visited earlier. Moreover, the margin metric can also be employed to rank trees based on their contribution to the overall margin—i.e., the certainty of the majority class—so that the trees contributing most to the margin are prioritized.

After ranking the DTs, our approach proceeds with computing the baseline number of trees required for classification. This is done by evaluating a portion of the test set T, denoted as T_{off}, and computing the average $\mu_{M_{T_{off}}}$ and the standard deviation $\sigma_{M_{T_{off}}}$ of the margin. These statistics are calculated by considering only correctly classified samples. Due to the definition of the margin, these values offer a direct statistical estimate of the number of trees needed for inference.

Based on this, the number of trees required during inference is computed as: $N^* = N \cdot (\mu_{M_{T_{off}}} + k \cdot \sigma_{M_{T_{off}}})$, where N is the total number of trees in the ensemble, and k is a configurable parameter. This parameter allows tuning the trade-off between model accuracy and inference latency: positive values of k increase the number of trees visited, potentially improving accuracy, while negative values reduce the number of trees, lowering latency.

After ranking the trees and computing $\mu_{M_{T_{off}}}$ and $\sigma_{M_{T_{off}}}$, the model is deployed on the target node together with the tree ranking and the computed statistical values. Finally, we highlight that the tree ranking and the confidence-level evaluation are independent and can be performed in any order.

B. After-Deployment Phase

After deploying the model on the target, the system is ready to be used in the field and to perform inferences using the stored RF classifier.

To this end, the early stopping criterion is applied by visiting trees according to the stored ranking order. After each tree is visited, the v vector is updated based on the class predicted by the most recently evaluated tree, and the number of visited trees n is incremented. This process continues until $n = N^*$. At that point, the prediction is stopped and a majority vote is applied to v to determine the most frequently predicted class.

Regarding N^*, this value can be dynamically updated on-board by adjusting the k parameter. This is particularly relevant in real-world IoT scenarios, where the classifier is used to predict future states and/or conditions of a system that evolves over time. In such contexts, k can be tuned based on the observed classification accuracy, determined by comparing the predicted and actual labels of samples.

Specifically, k can be increased when the model starts producing inaccurate classifications, as more computational effort may be required to correctly classify the current inputs. Conversely, when predictions consistently become accurate, the k value can be reduced to minimize latency and save energy.

IV. Experimental Evaluation

To validate the effectiveness of our proposed method, we considered three datasets from the UCI Machine Learning Repository [1]: Avila, Dry Bean, and AI4I. For each dataset, we trained a RF classifier composed of 100 trees, using five-fold cross-validation for hyperparameter tuning. Information regarding the considered dataset and the trained classifiers are reported in Table I.

TABLE I: Overview of the datasets used in the evaluation, including size, feature count, class count, and achieved accuracy.

Dataset	Instances	Attributes	Classes	Accuracy (%)
Avila	20,867	10	12	94.45
AI4I	10,000	6	2	98.87
Dry Bean	13,611	17	7	93.38

For each trained model, we applied the proposed early-stopping approach across five different runs, each with a distinct T_{off} split. In all cases, T_{off} was set to 50% of the full test set. Tree ranking was carried out using the MeanD-M algorithm. For each T_{off} split, we computed the mean $\mu_{M_{T_{\text{off}}}}$ and standard deviation $\sigma_{M_{T_{\text{off}}}}$, and stored them, along with the rank of trees, in a binary file. During the online-phase, this file, along with another one containing the original classifier, is loaded into the memory of an STM32H743ZI MCU. Indeed, during the experimental campaign, the considered MCU loads both the classifier and the corresponding early-stopping configuration parameters from memory to perform inference.

For each combination of dataset and early-stopping configuration, we submitted the remaining 50% of test samples not included in T_{off} to the inference process. Figures 4 and 5 report the average computational time and classification accuracy, respectively, for varying values of the k parameter. As expected, increasing k leads to longer inference times due to the inclusion of more decision trees in the classification

[1] https://archive.ics.uci.edu/

Authorized licensed use limited to: POLITECHNIKI WARSZAWSKIEJ. Downloaded on August 21,2026 at 17:14:48 UTC from IEEE Xplore. Restrictions apply.

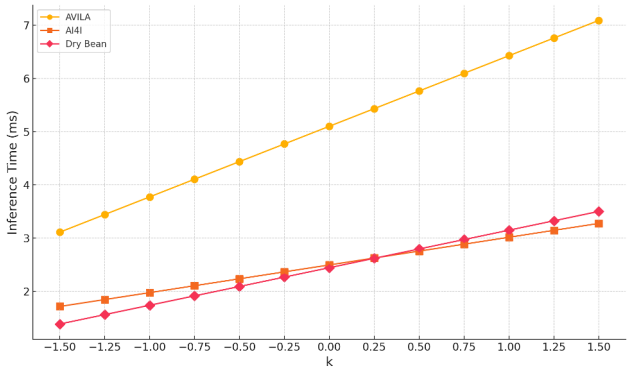


Fig. 4: Inference time of the three-different classifiers when varying the k-parameter.

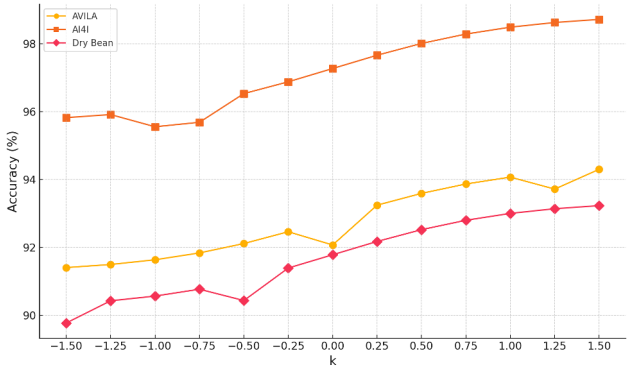


Fig. 5: Accuracy of the considered classifiers when varying the k value.

process. Correspondingly, the accuracy also improves as more trees contribute to the final decision.

Nevertheless, it is worth highlighting that a well-calibrated trade-off allows for a significant reduction in inference time with only a minimal impact on classification accuracy. On average, with a visit of only 53 trees, it is possible to achieve an accuracy loss of only 2%.

V. Conclusion

In conclusion, this paper presents an early-stopping approach for RF classifiers. The proposed method combines decision tree ranking with confidence-level estimation based on the margin metric. During the deployment, it enables configurable control over the number of trees evaluated at inference time. Experimental results demonstrate that the approach is effective and can deliver significant benefits in terms of computational efficiency, making it well-suited for execution on resource-constrained devices.

References

[1] P. Kaur, R. Kumar, and M. Kumar, "A healthcare monitoring system using random forest and internet of things (IoT)," *Multimedia Tools and Applications*, vol. 78, no. 14, pp. 19 905–19 916, Jul. 2019. [Online]. Available: https://doi.org/10.1007/s11042-019-7327-8

[2] M. Barbareschi, A. Emmanuele, N. Mazzocca, and F. Rocco di Torrepadula, "Designing on-board explainable passenger flow prediction," *Engineering Applications of Artificial Intelligence*, vol. 139, p. 109648, Jan. 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0952197624018062

[3] M. Barbareschi, S. Barone, A. Emmanuele, and N. Mazzocca, "Exploiting Functional Approximation on Decision-Tree Based Multiple Classifier Systems," in *2024 IFIP/IEEE 32nd International Conference on Very Large Scale Integration (VLSI-SoC)*, Oct. 2024, pp. 1–4, iSSN: 2324-8440. [Online]. Available: https://ieeexplore.ieee.org/document/10767841/?arnumber=10767841

[4] M. Barbareschi, V. Casola, A. Emmanuele, and D. Lombardi, "A Lightweight PUF-Based Protocol for Dynamic and Secure Group Key Management in IoT," *IEEE Internet of Things Journal*, vol. 11, no. 20, pp. 32 969–32 984, Oct. 2024, conference Name: IEEE Internet of Things Journal. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10614146

[5] E. Tabanelli, G. Tagliavini, and L. Benini, "Optimizing Random Forest-Based Inference on RISC-V MCUs at the Extreme Edge," vol. 41, no. 11, 2022.

[6] ——, "DNN is not all you need: Parallelizing Non-Neural ML Algorithms on Ultra-Low-Power IoT Processors," Nov. 2022, arXiv:2107.09448. [Online]. Available: http://arxiv.org/abs/2107.09448

[7] M. Barbareschi and S. Barone, "Investigating the Resilience Source of Classification Systems for Approximate Computing Techniques," *IEEE Transactions on Emerging Topics in Computing*, pp. 1–12, 2024, conference Name: IEEE Transactions on Emerging Topics in Computing. [Online]. Available: https://ieeexplore.ieee.org/document/10542568/?arnumber=10542568

[8] M. Barbareschi, S. Barone, and N. Mazzocca, "Advancing synthesis of decision tree-based multiple classifier systems: an approximate computing case study," *Knowledge and Information Systems*, vol. 63, no. 6, pp. 1577–1596, Jun. 2021. [Online]. Available: https://link.springer.com/10.1007/s10115-021-01565-5

[9] O. Sagi and L. Rokach, "Explainable decision forest: Transforming a decision forest into an interpretable tree," *Information Fusion*, vol. 61, pp. 124–138, Sep. 2020. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S1566253519307869

[10] F. Yang, W.-h. Lu, L.-k. Luo, and T. Li, "Margin optimization based pruning for random forest," *Neurocomputing*, vol. 94, pp. 54–63, Oct. 2012. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0925231212003396

[11] M. Barbareschi, "Implementing Hardware Decision Tree Prediction: A Scalable Approach," in *2016 30th International Conference on Advanced Information Networking and Applications Workshops (WAINA)*. Crans-Montana, Switzerland: IEEE, Mar. 2016, pp. 87–92. [Online]. Available: http://ieeexplore.ieee.org/document/7471178/

[12] M. Barbareschi, E. Battista, N. Mazzocca, and S. Venkatesan, "A hardware accelerator for data classification within the sensing infrastructure," in *Proceedings of the 2014 IEEE 15th International Conference on Information Reuse and Integration (IEEE IRI 2014)*. Redwood City, CA, USA: IEEE, Aug. 2014, pp. 400–405. [Online]. Available: http://ieeexplore.ieee.org/document/7051917/

[13] T. P. Dinh, C. Pham-Quoc, T. N. Thinh, B. K. D. Nguyen, and P. C. Kha, "A flexible and efficient FPGA-based random forest architecture for IoT applications," *Internet of Things*, vol. 22, p. 100813, Jul. 2023. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S2542660523001361

[14] C. Kim, J. Chhugani, N. Satish, E. Sedlar, A. D. Nguyen, T. Kaldewey, V. W. Lee, S. A. Brandt, and P. Dubey, "FAST: fast architecture sensitive tree search on modern CPUs and GPUs," in *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*, ser. SIGMOD '10. New York, NY, USA: Association for Computing Machinery, Jun. 2010, pp. 339–350. [Online]. Available: https://dl.acm.org/doi/10.1145/1807167.1807206

[15] B. Sudharsan, P. Patel, J. G. Breslin, and M. I. Ali, "Ultra-fast Machine Learning Classifier Execution on IoT Devices without SRAM Consumption," in *2021 IEEE International Conference on Pervasive Computing and Communications Workshops and other Affiliated Events (PerCom Workshops)*. Kassel, Germany: IEEE, Mar. 2021, pp. 316–319. [Online]. Available: https://ieeexplore.ieee.org/document/9431061/

Authorized licensed use limited to: POLITECHNIKI WARSZAWSKIEJ. Downloaded on August 21,2026 at 17:14:48 UTC from IEEE Xplore. Restrictions apply.